

EduGenie: AI-Powered Learning Assistant

Project Description

EduGenie is an AI-powered educational assistant designed to simplify learning through intelligent content generation and interactive educational support. The application enables students to ask academic questions, understand complex concepts, generate quizzes, summarize lengthy content, receive personalized learning roadmaps, analyze educational images, and interact with uploaded PDF documents using advanced Generative AI.

The project is developed using FastAPI, Google Gemini 2.5 Flash, LaMini-Flan-T5, HTML, CSS, JavaScript, and Jinja2. FastAPI provides a high-performance backend, while Google Gemini powers intelligent text generation, image understanding, and PDF analysis. LaMini-Flan-T5 is integrated as an additional lightweight language model for educational text processing. The application also uses PyMuPDF to extract text from uploaded PDF documents and Pillow for image processing.

EduGenie provides an intuitive and responsive web interface where students can interact with AI through multiple educational tools. The system emphasizes structured responses, personalized learning, and interactive study assistance, making it suitable for students, educators, and self-learners.

Scenarios

Scenario 1: Question Answering

A student wants to learn about oceans and rivers. They ask, "Which is the largest ocean?" EduGenie instantly provides an accurate answer along with additional educational context, helping the student understand the topic more effectively.

Scenario 2: Quiz Generation

A student studying the Pythagoras Theorem wants to assess their understanding. They select "Generate Quiz", and EduGenie creates topic-specific multiple-choice questions along with an answer key for self-evaluation.

Scenario 3: Learning Recommendation

A learner interested in SQL requests a structured learning roadmap. EduGenie generates a personalized learning path covering beginner, intermediate, and advanced topics, along with recommended resources and practice projects.

Scenario 4: PDF Analysis

A student uploads lecture notes in PDF format and asks, "Explain Normalization from this document." EduGenie extracts the content from the uploaded PDF and answers the question based only on the information available in that document.

Technical Architecture

Description

EduGenie follows a modular three-tier architecture designed to provide intelligent educational assistance through Generative AI. The application consists of a responsive frontend built using HTML, CSS, JavaScript, and Jinja2, a FastAPI backend that processes user requests and communicates with AI models, and an AI layer powered by Google Gemini 2.5 Flash and LaMini-Flan-T5.

The backend handles request validation, prompt engineering, image processing, PDF text extraction, and AI response generation. Uploaded images are processed using Pillow, while uploaded PDF documents are parsed using PyMuPDF before being sent to the AI model for contextual question answering. The generated responses are formatted in Markdown and displayed through a clean and interactive web interface.

The architecture ensures modularity, scalability, and maintainability by separating the frontend, backend, and AI services into independent components.

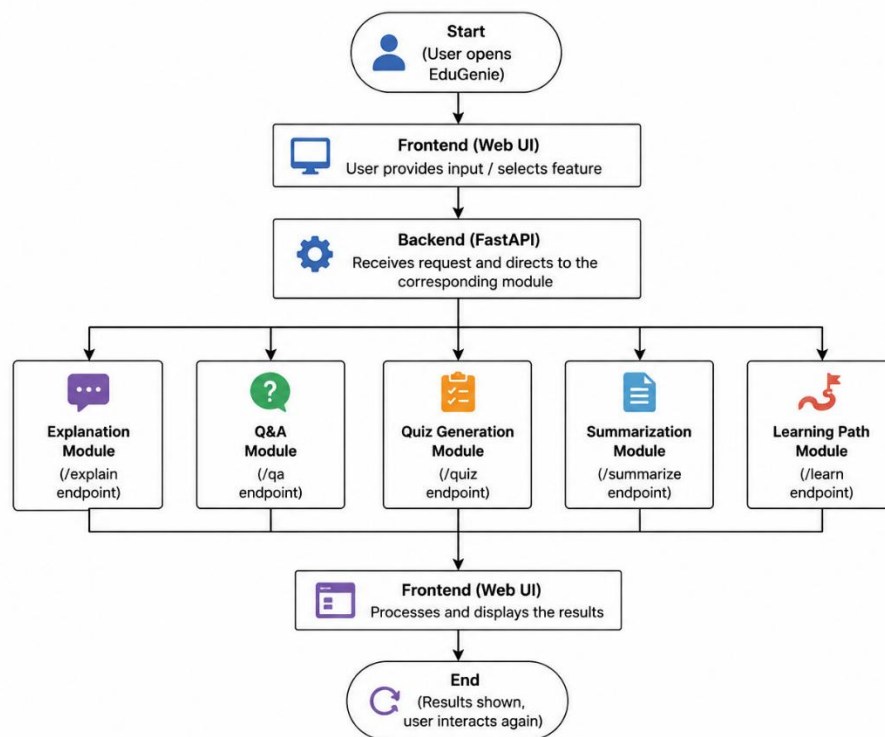


Figure: High-level architecture flow of EduGenie, from student request through the FastAPI backend to the Gemini and LaMini-Flan-T5 services and back.

Prerequisites

Before developing EduGenie, the following technologies and tools were required:

- Python Programming
- FastAPI Framework
- HTML, CSS, and JavaScript
- Google Gemini API
- LaMini-Flan-T5 (Transformers)
- Jinja2 Template Engine
- PyMuPDF
- Pillow
- Git and GitHub
- Visual Studio Code
- Postman (API Testing)

Technologies Used

Component	Technology
Programming Language	Python
Backend Framework	FastAPI
Frontend	HTML, CSS, JavaScript
AI Models	Google Gemini 2.5 Flash, LaMini-Flan-T5
Template Engine	Jinja2
PDF Processing	PyMuPDF
Image Processing	Pillow
API Testing	Postman
Version Control	Git & GitHub

Milestone 1: Model Selection and Architecture

In this milestone, the primary objective was to identify suitable Artificial Intelligence models and establish the overall architecture of the EduGenie application. The project required AI models capable of generating educational content, answering academic questions, explaining concepts, creating quizzes, analyzing uploaded images, and interacting with PDF documents. Along with model selection, the development environment, backend framework, frontend technologies, and application workflow were planned and configured to build a scalable AI-powered educational assistant.

Activity 1.1: Generate a Gemini API Key

Before integrating Artificial Intelligence into EduGenie, a Google Gemini API key was generated using Google AI Studio. This API key enables secure communication between the application and Google's Generative AI services.

Steps Followed

Step 1: Created a Google AI Studio account using a Google account.

Step 2: Generated a new Gemini API key from the Google AI Studio dashboard.

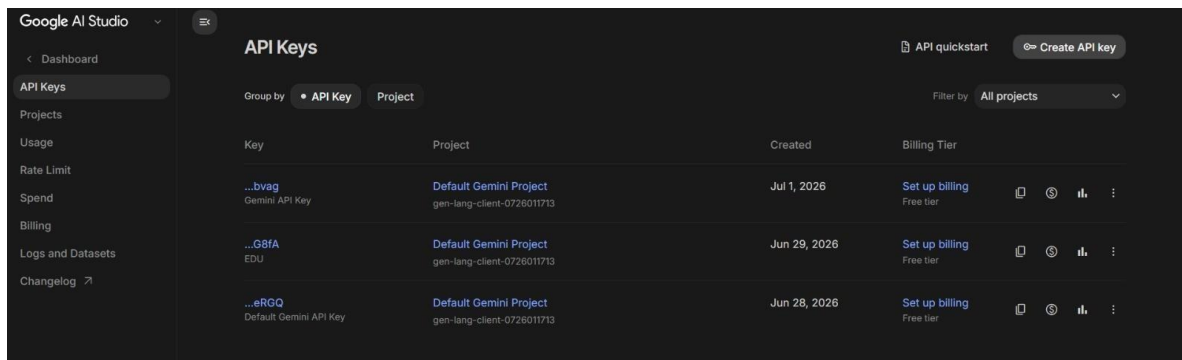


Figure: Google AI Studio — API Keys dashboard showing generated Gemini API keys.

Milestone 1: Model Selection and Architecture (contd.)

A new API key was created for the EduGenie project by selecting the imported Gemini project and submitting the creation form.

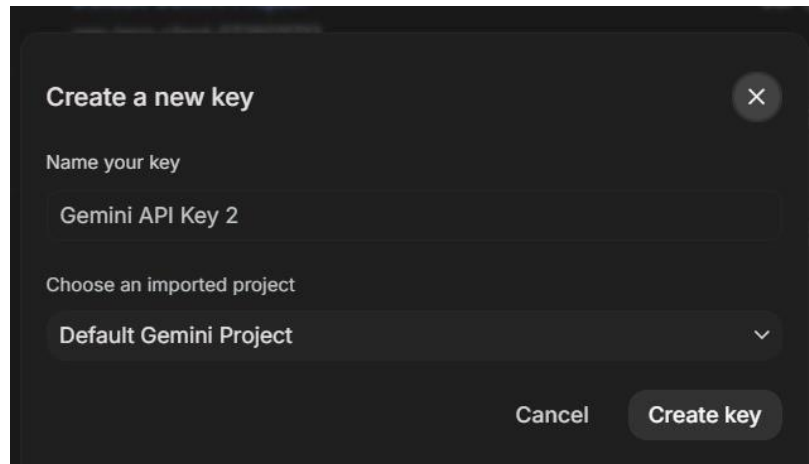


Figure: Creating a new Gemini API key inside Google AI Studio.

Step 3: Stored the generated API key securely inside the project's .env file.

```
GEMINI_API_KEY=your_api_key_here
```

Step 4: Loaded the environment variables using the python-dotenv library.

```
from dotenv import load_dotenv load_dotenv()
```

Step 5: Configured the Gemini API inside the project.

```
genai.configure(api_key=API_KEY)
```

Security Considerations: The .env file was excluded from GitHub using .gitignore to prevent accidental exposure of the API key.

Configuring the API Key in the Project

The Gemini API key was added to a .env file at the root of the EduGenie project inside Visual Studio Code, alongside the full project folder structure covering services, static assets, templates, and uploads.

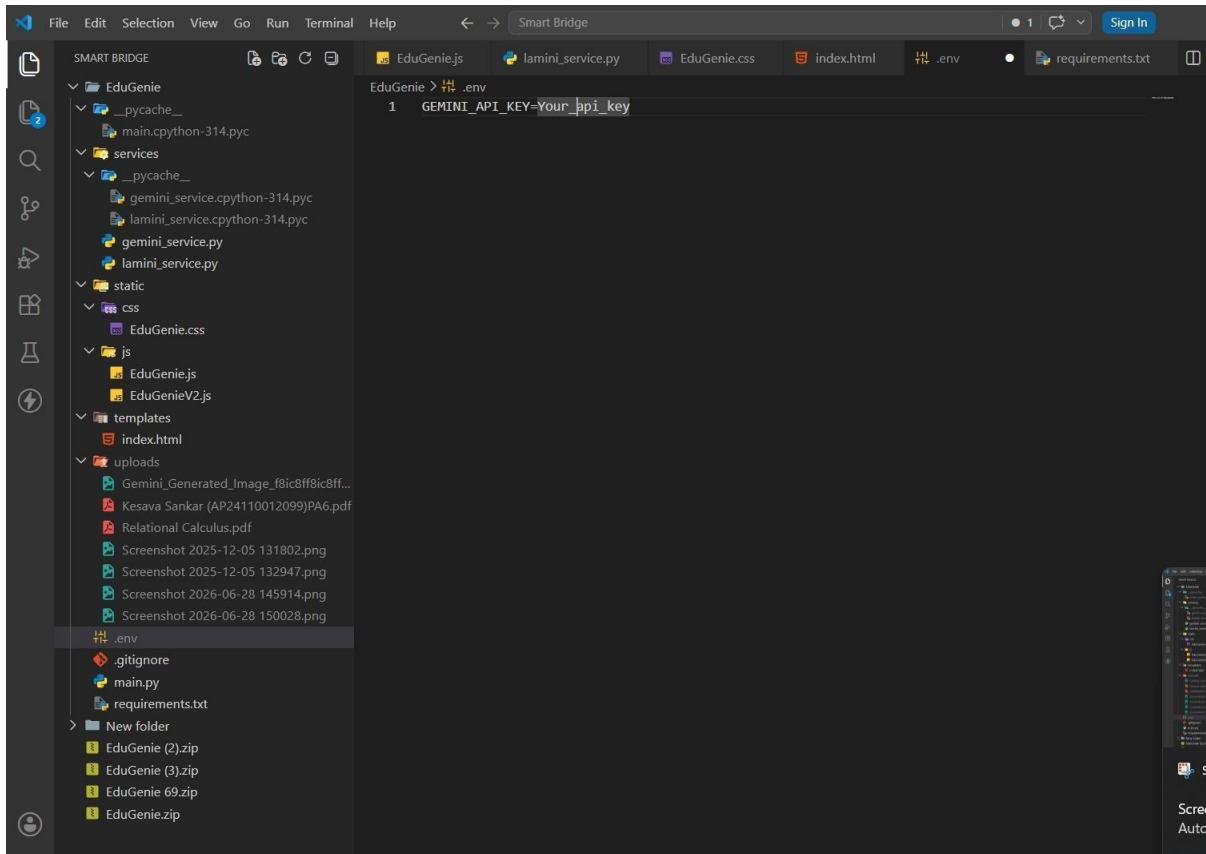


Figure: EduGenie project structure in VS Code showing the .env file with the GEMINI_API_KEY entry.

Activity 1.2: Research and Select the Appropriate AI Models

The EduGenie application required AI models capable of handling multiple educational tasks such as:

- Question Answering
- Topic Explanation
- Quiz Generation
- Learning Recommendation
- Image Analysis
- PDF Question Answering
- Educational Text Processing

After evaluating various available models, the following models were selected.

Google Gemini 2.5 Flash

Google Gemini 2.5 Flash was selected as the primary AI model because it provides:

- Fast response generation
- High-quality educational content

- Multimodal capabilities (text and image understanding)
- Large context window
- Easy integration through the Google Generative AI SDK Gemini powers the following modules:
- Question Answering
- Topic Explanation
- Quiz Generation
- Learning Recommendation
- Image Analysis
- PDF Analysis

```
model = genai.GenerativeModel("gemini-2.5-flash")
```

Figure: Gemini 2.5 Flash model initialization used across EduGenie's services.

LaMini-Flan-T5

LaMini-Flan-T5 was integrated as an additional lightweight language model to support educational text generation and demonstrate integration of multiple Generative AI models within the project.

Why These Models Were Selected

Model	Purpose
Google Gemini 2.5 Flash	Educational question answering, explanations, quizzes, image analysis
LaMini-Flan-T5	Lightweight educational text generation and experimentation with local

, PDF interaction, and
transformer-based models

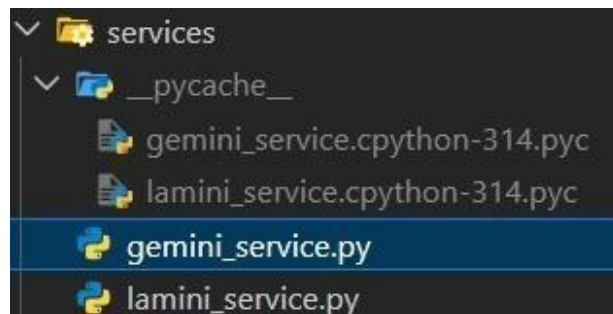


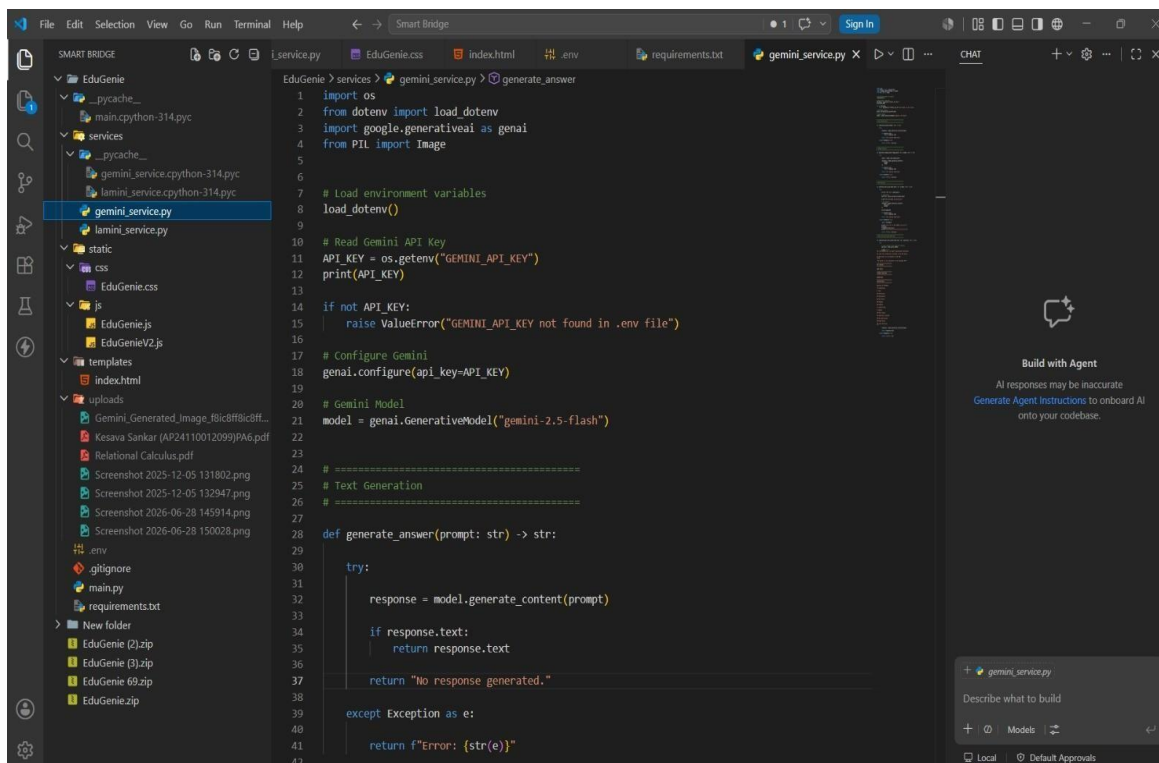
Figure: services folder containing gemini_service.py and lamini_service.py, the two AI integration modules.

Milestone 2: Core AI Service Development

This milestone focused on implementing the individual AI service modules that power EduGenie's educational features. Two dedicated service files were created inside the services/ package: `gemini_service.py` for Google Gemini text and vision generation, and `lamini_service.py` for local LaMini-Flan-T5 based text generation.

Activity 2.1: Implementing `gemini_service.py`

The Gemini service loads the API key from the environment, configures the Generative AI SDK, and exposes a `generate_answer()` function that sends a prompt to the gemini-2.5-flash model and returns the generated text, with error handling for missing keys or failed generations.



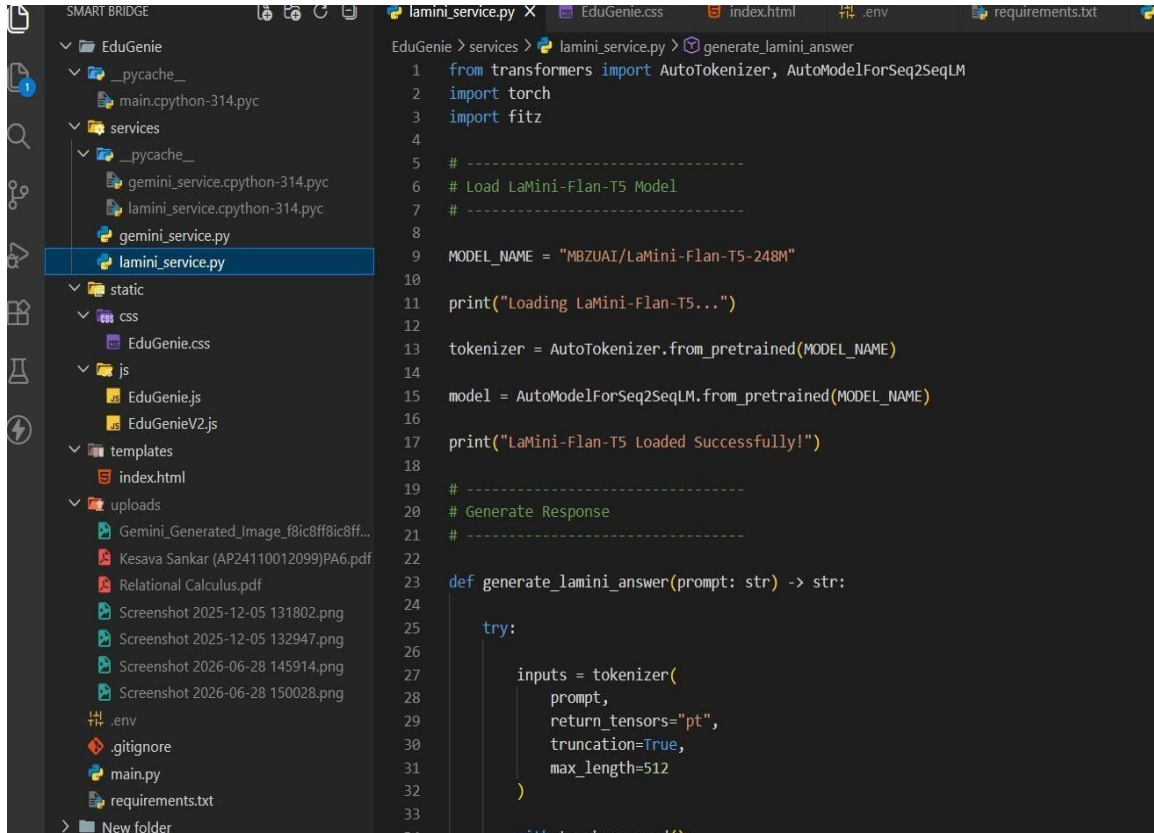
```

1  import os
2  from dotenv import load_dotenv
3  import google.generativeai as genai
4  from PIL import Image
5
6
7  # Load environment variables
8  load_dotenv()
9
10 # Read Gemini API Key
11 API_KEY = os.getenv("GEMINI_API_KEY")
12 print(API_KEY)
13
14 if not API_KEY:
15     raise ValueError("GEMINI_API_KEY not found in .env file")
16
17 # Configure Gemini
18 genai.configure(api_key=API_KEY)
19
20 # Gemini Model
21 model = genai.GenerativeModel("gemini-2.5-flash")
22
23
24 # =====
25 # Text Generation
26 # =====
27
28 def generate_answer(prompt: str) -> str:
29
30     try:
31         response = model.generate_content(prompt)
32
33         if response.text:
34             return response.text
35
36         return "No response generated."
37
38     except Exception as e:
39         return f"Error: {str(e)}"
40
41
42
  
```

Figure: `gemini_service.py` — loading the API key, configuring Gemini, and implementing `generate_answer()`.

Activity 2.2: Implementing `lamini_service.py`

The LaMini service loads the MBZUAI/LaMini-Flan-T5-248M model and tokenizer using the Hugging Face Transformers library, then defines a `generate_lamini_answer()` function that tokenizes the input prompt, runs it through the model, and decodes the generated tokens into a readable educational response.



```

1  from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
2  import torch
3  import fitz
4
5  # -----
6  # Load LaMini-Flan-T5 Model
7  # -----
8
9  MODEL_NAME = "MBZUAI/LaMini-Flan-T5-248M"
10
11 print("Loading LaMini-Flan-T5...")
12
13 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
14
15 model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
16
17 print("LaMini-Flan-T5 Loaded Successfully!")
18
19 # -----
20 # Generate Response
21 # -----
22
23 def generate_lamini_answer(prompt: str) -> str:
24     try:
25         inputs = tokenizer(
26             prompt,
27             return_tensors="pt",
28             truncation=True,
29             max_length=512
30         )
31         with torch.no_grad():
32

```

Figure: lamini_service.py — loading the LaMini-Flan-T5 model and tokenizer, and defining the generation function.

Gemini Text Generation Logic

The generate_answer() function forms the core of EduGenie's question-answering, explanation, quiz generation, and learning recommendation features. It sends the constructed prompt directly to the Gemini model and safely handles cases where no response text is returned.

```
PS C:\SRM Things\full stack\Smart Bridge\EduGenie> py -m uvicorn main:app --reload

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
AQ.Ab8RN6KXY_u6dybWdYGMiFgVRu9B0wwOmi4Vzy7xCNRkrbgvbg
Loading LaMini-Flan-T5...
Loading weights: 100%|████████████████████████████████████████| 284/284 [00:00<00:00, 47086.03it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to l
m_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them
. You should update the config with `tie_word_embeddings=False` to silence this warning.
LaMini-Flan-T5 Loaded Successfully!
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [7680] using StatReload
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable high
er rate limits and faster downloads.
C:\SRM Things\full stack\Smart Bridge\EduGenie\services\gemini_service.py:3: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
AQ.Ab8RN6KXY_u6dybWdYGMiFgVRu9B0wwOmi4Vzy7xCNRkrbgvbg
Loading LaMini-Flan-T5...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable high
er rate limits and faster downloads.
Loading weights: 100%|████████████████████████████████████████| 284/284 [00:00<00:00, 35544.95it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to l
m_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them
. You should update the config with `tie_word_embeddings=False` to silence this warning.
LaMini-Flan-T5 Loaded Successfully!
INFO:      Started server process [21720]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Figure: Full implementation of gemini_service.py including API key validation, model configuration, and the generate_answer() function with exception handling.

Activity 2.3: Managing Project Dependencies

All required Python packages for EduGenie were declared inside requirements.txt, covering the FastAPI backend, Gemini SDK, LaMini-Flan-T5 model dependencies, PDF and image processing libraries, and supporting utilities.

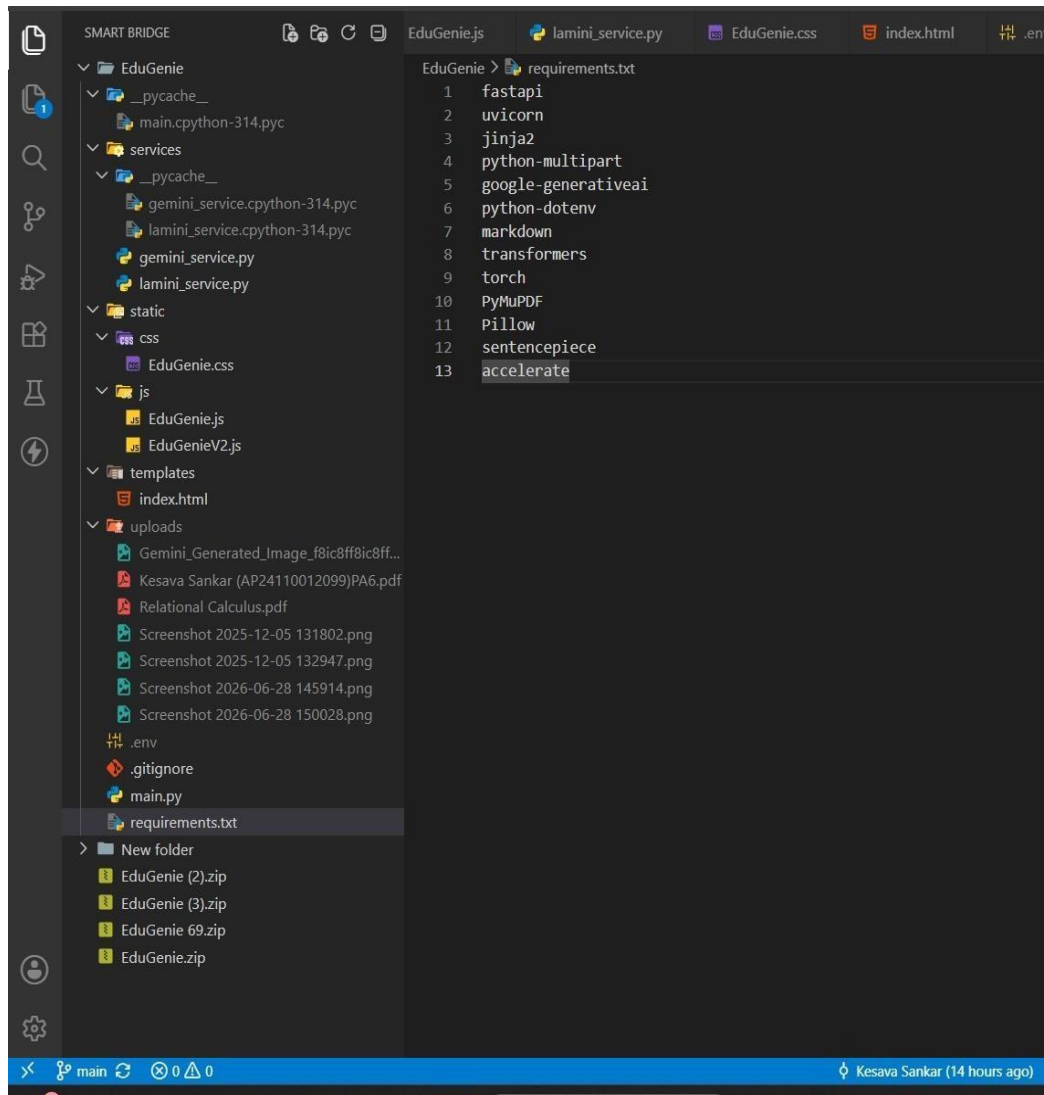


Figure: requirements.txt listing FastAPI, Uvicorn, Jinja2, python-multipart, google-generativeai, python-dotenv, markdown, transformers, torch, PyMuPDF, Pillow, sentencepiece, and accelerate.

Milestone 2: Core AI Service Development (contd.)

The following activities expand on EduGenie's remaining core functionalities — Quiz Generation, Learning Recommendation, Image Analysis, PDF Analysis, Summary Generation, and the overall integration of Google Gemini AI services — along with the corresponding implementation details and supporting code.

Activity 2.4: Quiz Generation Module

The Quiz Generation module automatically creates multiple-choice questions based on a topic selected by the student. When a student chooses the “Generate Quiz” option, the FastAPI backend constructs a structured prompt asking Gemini to produce a fixed number of multiple-choice questions, each with four options and a clearly marked correct answer. The same `generate_answer()` function used for question answering is reused here, with a specialised prompt template, which keeps the codebase modular and avoids duplicating Gemini integration logic. The generated quiz is returned in Markdown format so that questions, options, and the answer key are displayed clearly on the frontend, allowing students to self-evaluate their understanding of the topic.

```
def generate_answer(prompt: str) -> str:
    try:
        response = model.generate_content(prompt)

        if response.text:
            return response.text

        return "No response generated."

    except Exception as e:
        return f"Error: {str(e)}"
```

Figure: `generate_answer()` in `gemini_service.py` — the shared Gemini text-generation function reused by the Quiz Generation and Learning Recommendation modules.

Activity 2.5: Learning Recommendation Module

The Learning Recommendation module builds a structured, multi-level roadmap for any topic entered by the student. The backend sends a prompt to Gemini instructing it to organise the response into Beginner, Intermediate, and Advanced sections. Each level includes the key concepts to be learned, along with recommended resources and hands-on practice projects appropriate to that stage. This gives students a clear, progressive learning path rather than a single flat answer, and encourages self-paced learning by

breaking a broad topic into manageable milestones. The roadmap is rendered in Markdown so headings, bullet points, and resource links remain readable inside the chat interface.

Activity 2.6: Image Analysis Module

EduGenie supports visual question answering using the Gemini Vision model. A dedicated `/image` endpoint accepts an uploaded image file together with a text prompt describing what the student wants to know. The backend first ensures an uploads directory exists, saves the incoming image to disk using an asynchronous file write, and then passes both the saved image path and the prompt to the `generate_image_answer()` helper function. This function opens the image using Pillow, sends it together with the prompt to the Gemini model, and returns the model's textual explanation, which is converted to HTML using the markdown library before being sent back to the frontend. This allows students to upload diagrams, handwritten problems, screenshots of textbook pages, or charts and ask natural-language questions directly about their contents.

```
@app.post("/image")
async def image_analysis(
    image: UploadFile = File(...),
    prompt: str = Form(...)
):
    # Create uploads folder if it doesn't exist
    os.makedirs("uploads", exist_ok=True)

    image_path = os.path.join("uploads", image.filename)

    # Save uploaded image
    with open(image_path, "wb") as buffer:
        buffer.write(await image.read())

    # Ask Gemini Vision
    answer = generate_image_answer(image_path, prompt)

    formatted_answer = markdown.markdown(answer)

    return {
        "response": formatted_answer
    }
```

Figure: The `/image` FastAPI route — saving the uploaded file and forwarding it to `generate_image_answer()` for Gemini Vision analysis.

```
def generate_image_answer(image_path: str, prompt: str) -> str:
    try:
        image = Image.open(image_path)

        response = model.generate_content([
            prompt,
            image
        ])

        if response.text:
            return response.text

        return "No response generated."

    except Exception as e:
        return f"Error: {str(e)}"
```

Figure: `generate_image_answer()` in `gemini_service.py` — opening the image with Pillow and calling the Gemini vision-capable model.

Activity 2.7: PDF Analysis Module

The PDF Analysis module allows students to upload study material in PDF format and ask questions answered strictly from that document. The `generate_pdf_answer()` function uploads the PDF file directly to the Gemini File API using `genai.upload_file()`, then sends both the uploaded file reference and the student's prompt to the Gemini model in a single `generate_content()` call. Exception handling captures and logs any errors encountered while reading or processing the PDF, printing a detailed traceback to the server console for easier debugging, while still returning a clean, user-friendly error message to the frontend if something goes wrong. This design allows EduGenie to answer questions about lecture notes, textbooks, or assignment sheets without requiring any separate text-extraction step in application code, since Gemini itself parses the uploaded PDF.


```
def generate_pdf_answer(pdf_path: str, prompt: str) -> str:

    try:

        print(f"PDF Path: {pdf_path}")

        # Upload PDF
        pdf_file = genai.upload_file(pdf_path)

        print("PDF Uploaded Successfully")

        # Ask Gemini
        response = model.generate_content([
            pdf_file,
            prompt
        ])

        print(response)

        if response.text:
            return response.text

        return "No response generated."

    except Exception as e:

        import traceback

        print("\n===== PDF ERROR =====")
        print(e)
        traceback.print_exc()
        print("=====\n")

        return f"Error: {str(e)}"
```

Figure: `generate_pdf_answer()` in `gemini_service.py` — uploading the PDF to Gemini and generating a context-aware answer, with full error tracebacks for debugging.

Activity 2.8: Summary Generation

EduGenie can condense lengthy text into concise, easy-to-revise notes. A student pastes a large block of text — such as lecture notes or an article — and selects the Summarize task. The backend sends the content to Gemini with a prompt requesting a shorter version that retains only the key points, definitions, and takeaways. This feature helps students revise quickly before exams by turning verbose material into a compact set of highlights, and it reuses the same `generate_answer()` pipeline as the other text-based modules, keeping the implementation consistent and lightweight.

Activity 2.9: Integration of Gemini AI Services

All of EduGenie's AI-powered features are unified through a single, securely configured Gemini client. On startup, the application loads environment variables using `python-dotenv`, reads the `GEMINI_API_KEY` from the `.env` file, and raises a clear error if the key is missing so misconfiguration is caught immediately rather than failing silently later. Once validated, the key is used to configure the `google-generativeai` SDK, and a single `gemini-2.5-flash` model instance is created and shared across the question answering, explanation,

quiz generation, learning recommendation, image analysis, and PDF analysis routes. Keeping this configuration in one place, inside `gemini_service.py`, ensures that future changes — such as switching models or rotating API keys — only need to be made in one file.

```
# Load environment variables
load_dotenv()

# Read Gemini API Key
API_KEY = os.getenv("GEMINI_API_KEY")
print(API_KEY)

if not API_KEY:
    raise ValueError("GEMINI_API_KEY not found in .env file")

# Configure Gemini
genai.configure(api_key=API_KEY)

# Gemini Model
model = genai.GenerativeModel("gemini-2.5-flash")
```

Figure: Loading the Gemini API key from .env, validating it, and configuring the google-generativeai SDK with the gemini-2.5-flash model.

Milestone 3: FastAPI Backend Development

Milestone 3 focused on building `main.py`, the central FastAPI application that ties together the frontend templates and the Gemini and LaMini AI services.

Activity 3.1: Defining Application Routes

- A home route (/) renders the main `index.html` interface using Jinja2 templates.
- Dedicated POST routes handle question answering, quiz generation, topic explanation, learning recommendations, image analysis, and PDF-based question answering.
- Uploaded images are processed using Pillow before being passed to the Gemini vision model.
- Uploaded PDF documents are parsed using PyMuPDF to extract text, which is then included as context in the prompt sent to Gemini.

Activity 3.2: Connecting Services to Routes

- Each route imports the relevant function from `gemini_service.py` or `lamini_service.py`.
- User input is validated before being forwarded to the AI service.
- Responses are formatted in Markdown and returned to the frontend for rendering.
- Errors from the AI services are caught and returned as user-friendly messages.

Activity 3.3: Running the Development Server

The FastAPI application was started locally using Uvicorn with hot-reload enabled, allowing changes to the codebase to be reflected immediately during development.

```
PS C:\SRM Things\full stack\Smart Bridge> cd EduGenie
PS C:\SRM Things\full stack\Smart Bridge\EduGenie> py -m uvicorn main:app --reload
```

Figure: Navigating to the EduGenie project directory and starting the server with `py -m uvicorn main:app --reload`.

```
PS C:\SRM Things\full stack\Smart Bridge\EduGenie\py -m uvicorn main:app --reload

All support for the 'google.generativeai' package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
AQ_AbBRt6KQKY UedybWdYGMiGfVRU9B0wOmI4Vzy7XCNrkrgbvag
Loading LaMini-Flan-T5...
Loading weights: 100% [redacted] 284/284 [00:00<00:00, 47886.03it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to l
m_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them
. You should update the config with 'tie_word_embeddings=False' to silence this warning.
LaMini-Flan-T5 loaded Successfully!
INFO: uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [7680] using StatReload
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable high
er rate limits and faster downloads.
C:\SRM Things\full stack\Smart Bridge\EduGenie\services\gemini_service.py:3: FutureWarning:

All support for the 'google.generativeai' package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
AQ_AbBRt6KQKY UedybWdYGMiGfVRU9B0wOmI4Vzy7XCNrkrgbvag
Loading LaMini-Flan-T5...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable high
er rate limits and faster downloads.
Loading weights: 100% [redacted] 284/284 [00:00<00:00, 35544.95it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to l
m_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them
. You should update the config with 'tie_word_embeddings=False' to silence this warning.
LaMini-Flan-T5 loaded Successfully!
INFO: Started server process [21720]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Server Startup and Model Loading

On startup, the LaMini-Flan-T5 weights are downloaded and loaded into memory, after which Uvicorn confirms the application is running and ready to accept requests on <http://127.0.0.1:8000>.

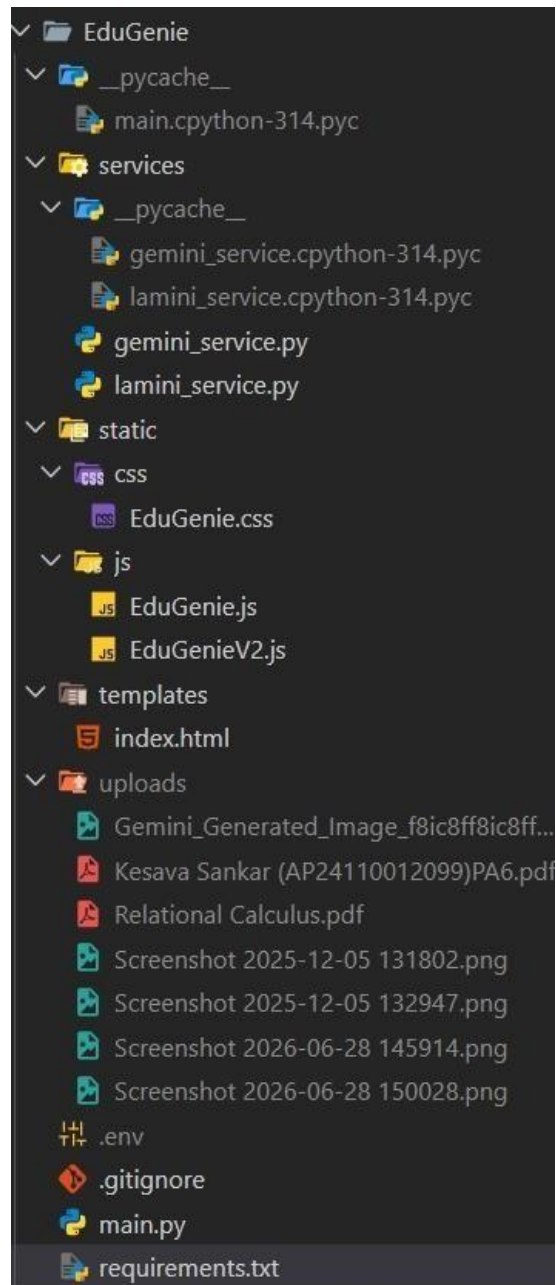


Figure: Terminal log showing LaMini-Flan-T5 weights loading successfully and Uvicorn confirming the application startup is complete.

Project Folder Structure

The EduGenie project is organized into clear, modular directories separating AI services, static assets, HTML templates, and uploaded files, making the codebase easy to navigate and extend.

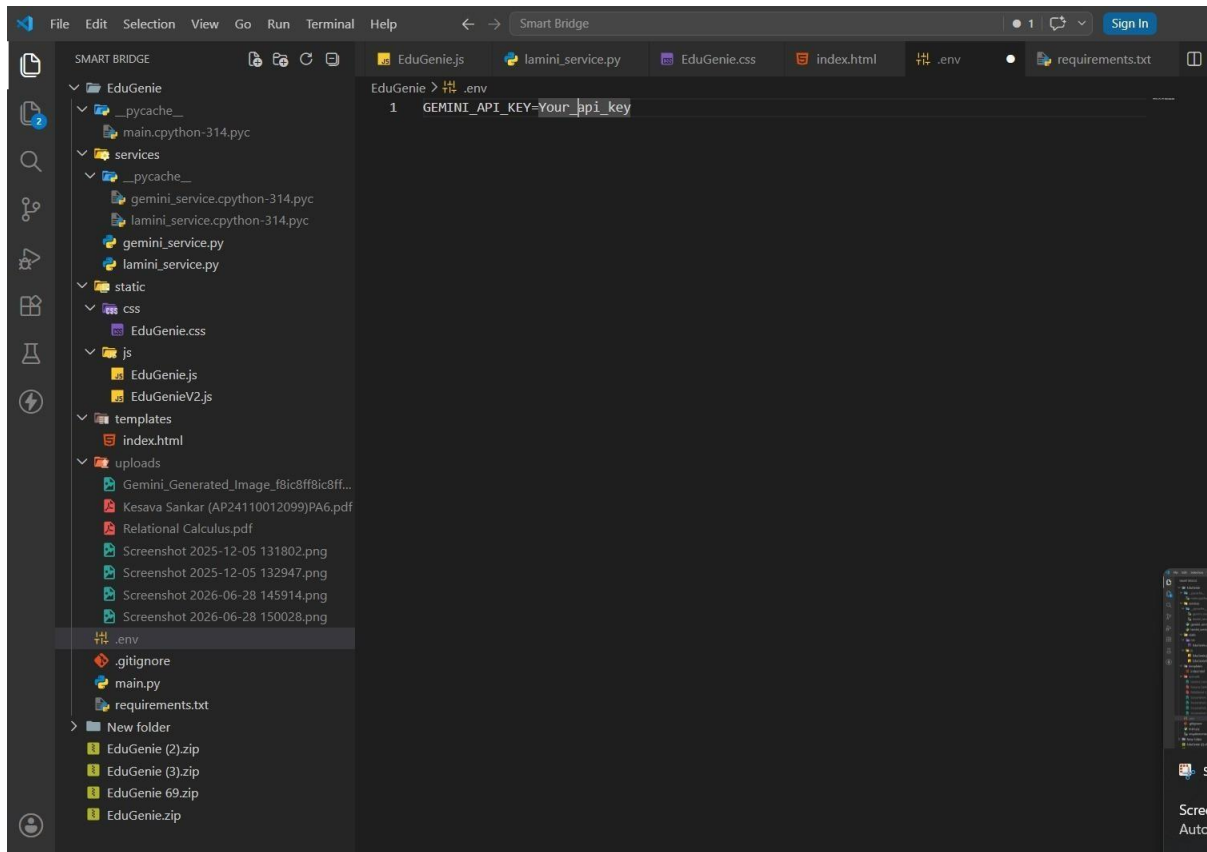


Figure: Full EduGenie directory tree, including services/, static/css, static/js, templates/, and uploads/.

Milestone 4: Frontend Development

Milestone 4 focused on building a clean, responsive user interface for EduGenie using HTML, CSS, and JavaScript, rendered through Jinja2 templates.

Activity 4.1: Building the Interface

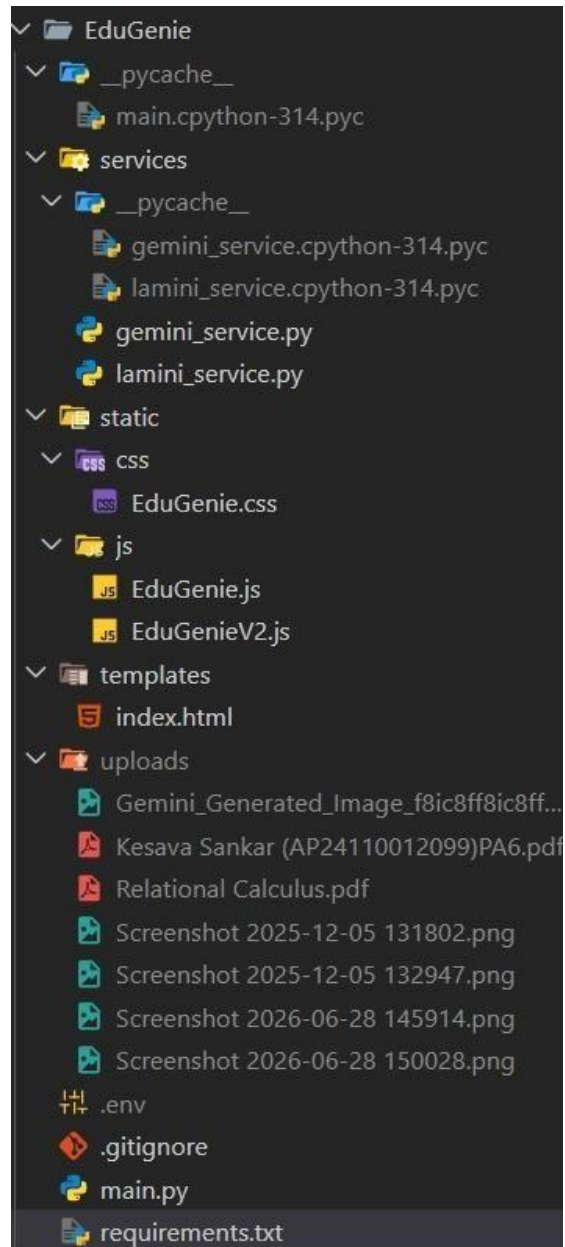
- index.html serves as the single-page interface with sections for question answering, quiz generation, learning recommendations, image upload, and PDF upload.
- EduGenie.css defines the visual styling, ensuring a consistent and modern look across all sections.
- EduGenie.js and EduGenieV2.js handle form submissions, asynchronous requests to the FastAPI backend, and dynamic rendering of AI-generated responses.

Activity 4.2: Handling File Uploads

- The interface allows students to upload images for the Image Analysis feature and PDF documents for the PDF Analysis feature.
- Uploaded files are sent to the backend using multipart form submissions handled by python-multipart.
- Generated responses, including quizzes, explanations, and PDF-based answers, are rendered dynamically on the page without a full page reload.

Complete Project File Tree

The diagram below summarizes the complete EduGenie project structure as viewed in the development environment, covering all core application files and supporting assets.



Milestone 4: Frontend Development (contd.)

Building on the frontend overview described earlier, the following activities detail the visual design, styling, scripting, and testing work carried out to give EduGenie a polished, responsive user interface.

Activity 4.1: User Interface Design

The EduGenie interface was designed around a single landing page that clearly communicates what the assistant can do before the student starts typing. A hero section introduces the product name and tagline, followed by five feature cards — Ask Questions, Explain Concepts, Summarize Notes, Generate Quiz, and Learning Roadmap — each with its own icon and short description, giving students an immediate overview of the available tools.

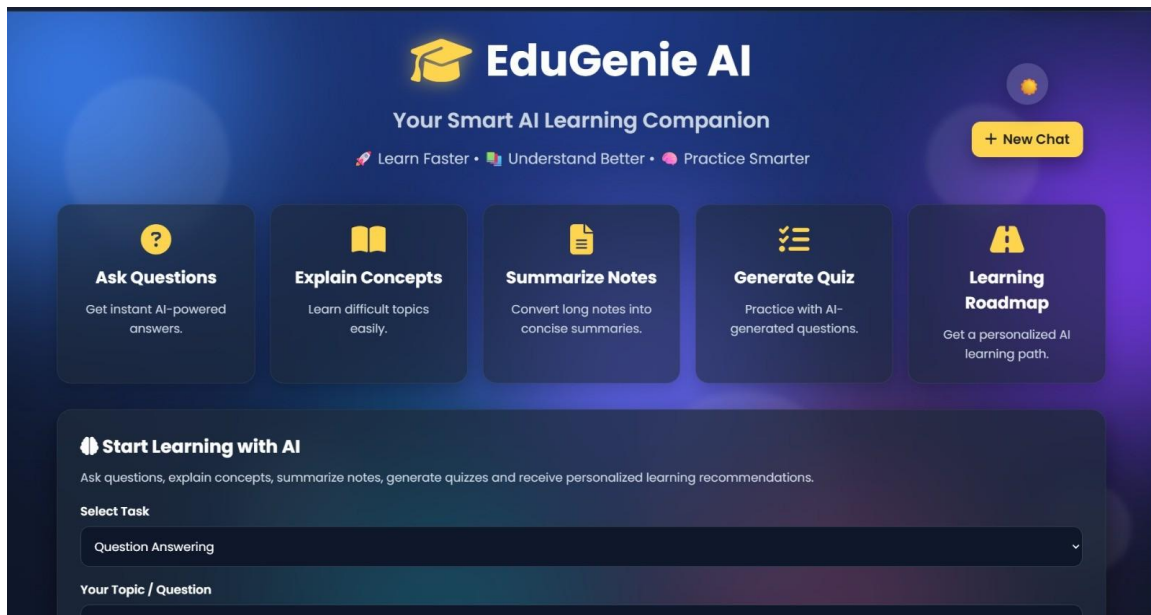


Figure: EduGenie AI landing page hero section with the five core feature cards.

Below the feature cards, a task selector lets the student choose which module to use (Question Answering, Topic Explanation, Summarization, Quiz Generation, or Learning Roadmap), alongside an input box, dedicated Upload Image and Upload PDF buttons, and a Generate action button. This layout keeps every core capability reachable from a single screen without additional navigation.

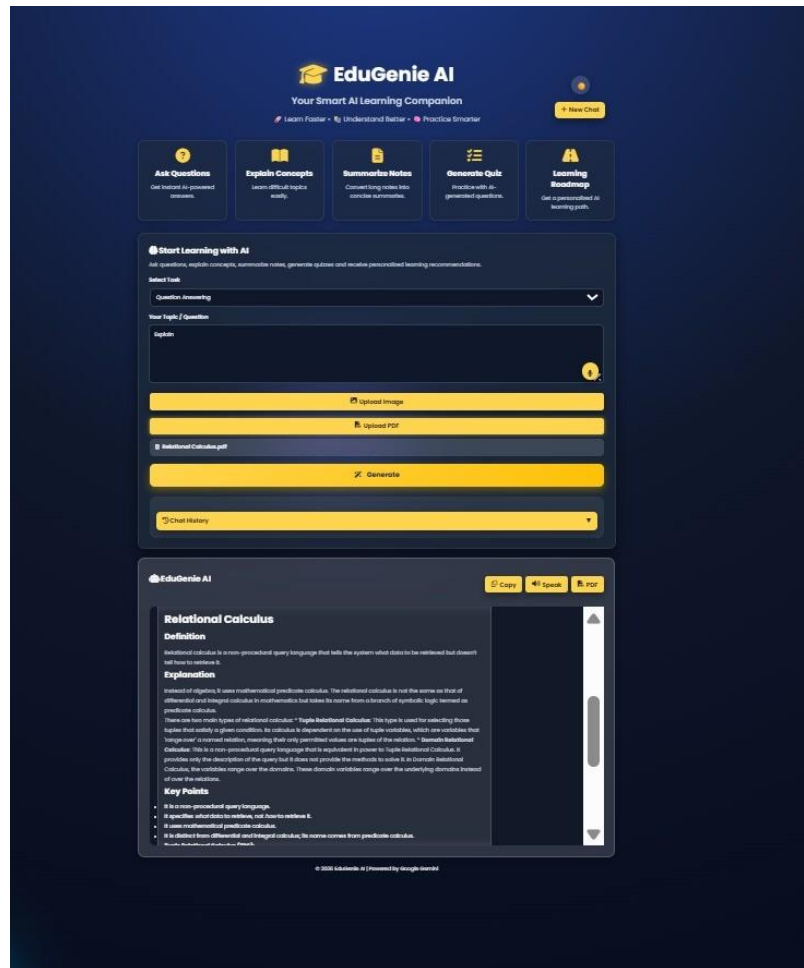


Figure: EduGenie AI task panel showing task selection, PDF upload, and a Gemini-generated explanation of Relational Calculus rendered in the response panel.

The interface and its supporting assets are organised inside the project's templates and static folders, with index.html rendered through Jinja2 and linked to the EduGenie.css stylesheet and the EduGenie.js / EduGenieV2.js scripts, keeping markup, styling, and behaviour cleanly separated.

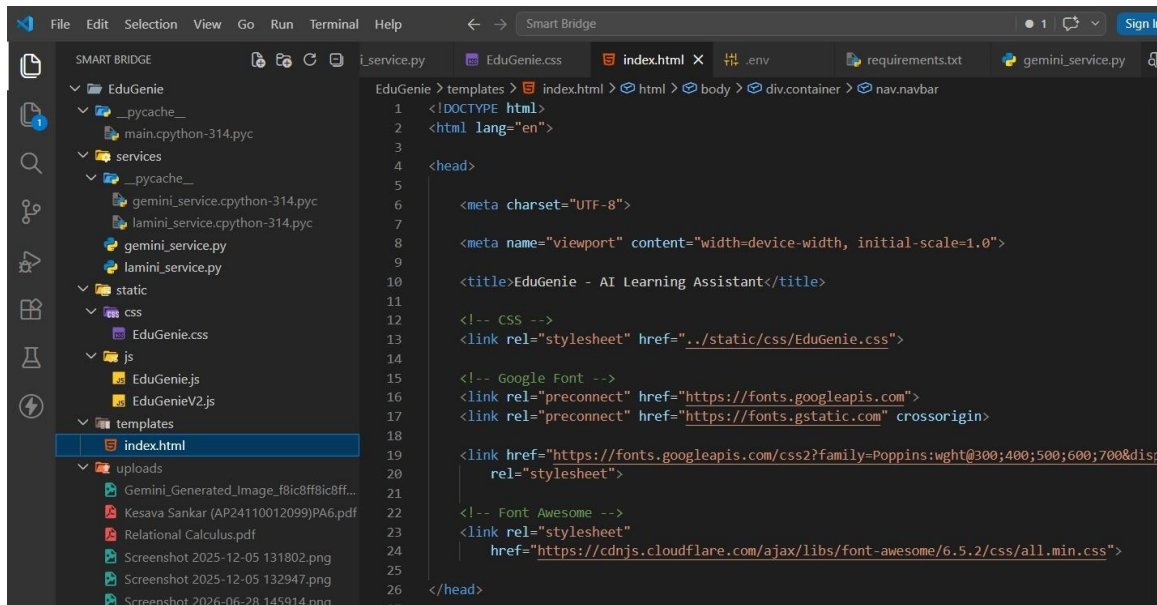


Figure: VS Code project view showing index.html linked to EduGenie.css, Google Fonts, and Font Awesome inside the head section of the template.

Activity 4.2: EduGenie AI Branding

A consistent visual identity was designed for EduGenie AI, combining a graduation-cap icon with a bold wordmark and the tagline “Your Smart AI Learning Companion.” This branding banner appears at the top of the interface and reinforces the assistant's educational purpose at a glance.



Figure: EduGenie AI brand banner with logo, name, and tagline.

Activity 4.3: CSS Styling

CSS was used extensively to give EduGenie a modern, dark-themed learning environment. A global reset removes default browser margins and paddings and applies the Poppins font family across the application. The page background uses a diagonal linear gradient moving through indigo, blue, and cyan tones, while the main container is centred, capped at a maximum width for readability on large screens, and layered above the background using z-index. Buttons, input fields, cards, the navigation header, and response

sections were all styled individually to create a cohesive, responsive interface that adapts cleanly across different screen sizes.

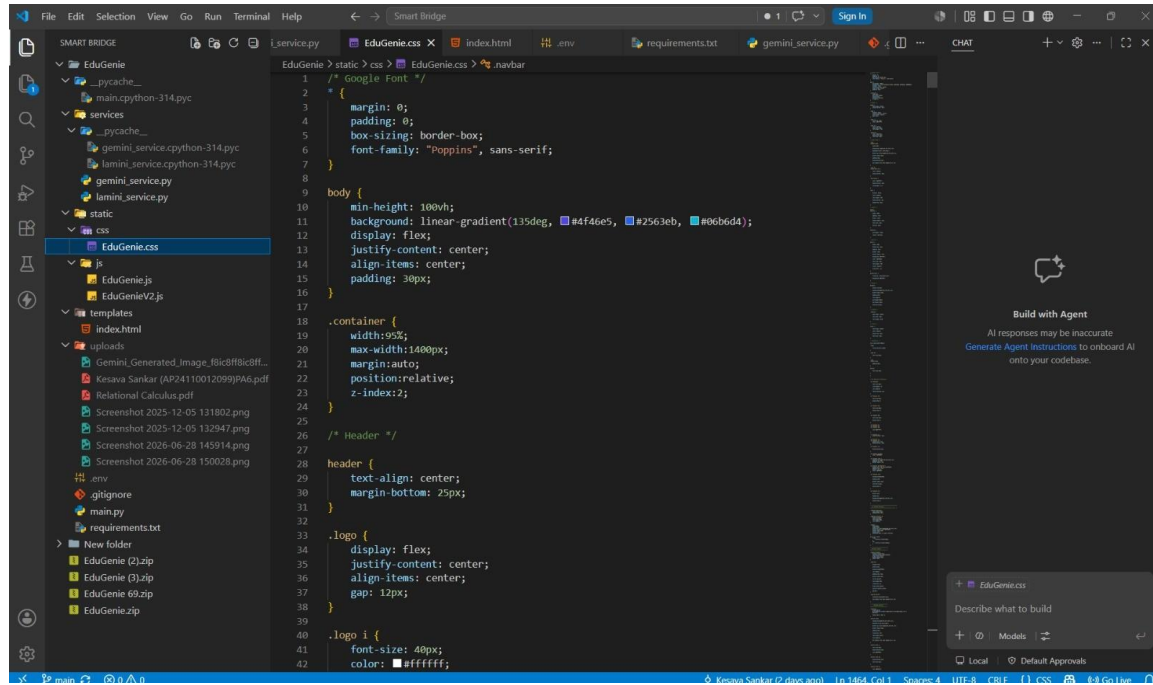


Figure: EduGenie.css — global reset, Poppins font family, gradient background, and container layout rules.

Activity 4.4: JavaScript Integration

JavaScript brings the interface to life without requiring page reloads. The `generateResponse()` function in `EduGenieV2.js` reads the student's input, immediately renders their message as a chat bubble with a timestamp, and then displays an animated "Thinking..." loading indicator while the request is in flight. Depending on which task is selected, the script builds the appropriate request — for image analysis it constructs a `FormData` object containing the selected image file and the prompt text and sends it as a multipart POST request to the `/image` endpoint. Similar branches handle PDF uploads and plain text requests to their respective backend routes. Once a response arrives, the loading bubble is replaced with the formatted AI answer, keeping the conversation feel fluid and chat-like.

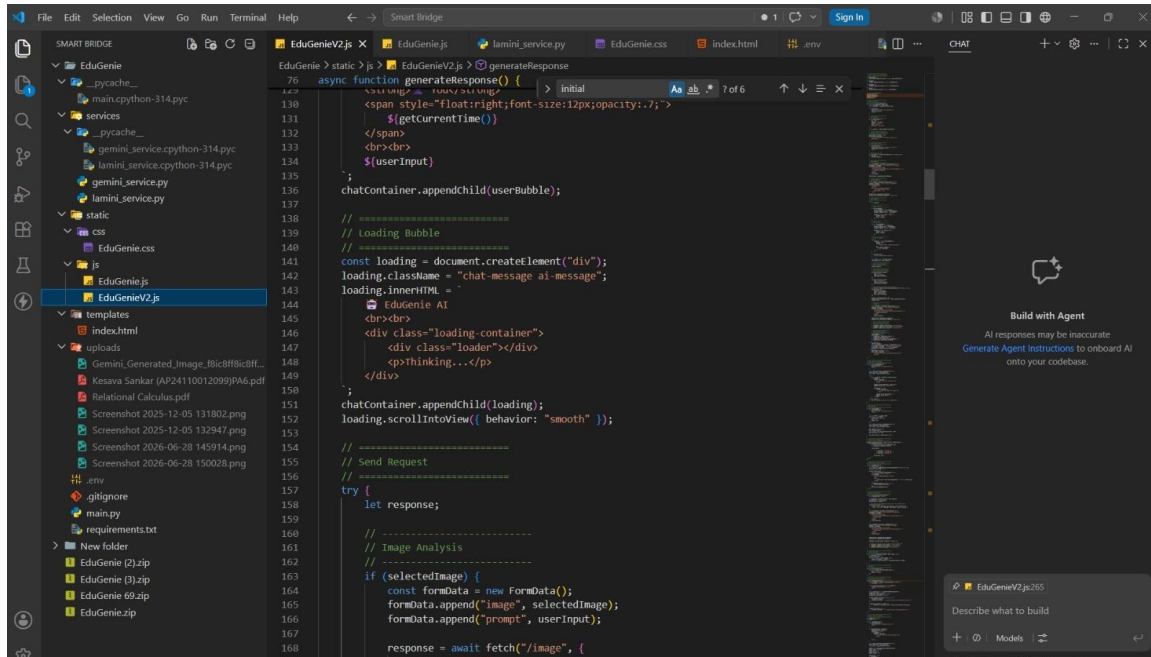


Figure: EduGenieV2.js — generateResponse() rendering the user bubble and loading indicator, then branching to handle image uploads via the /image endpoint.

Activity 4.5: Frontend Testing

Once styling and scripting were complete, the interface was tested end-to-end from a fresh page load. The welcome screen greets the student, lists the available capabilities, and exposes Copy, Speak, and PDF action buttons above every AI response, letting students copy an answer to the clipboard, have it read aloud, or export it as a PDF for offline revision. Each module was verified by submitting valid inputs and confirming that responses rendered correctly, with particular attention paid to image uploads, PDF uploads, and loading states to ensure a smooth, error-free user experience.

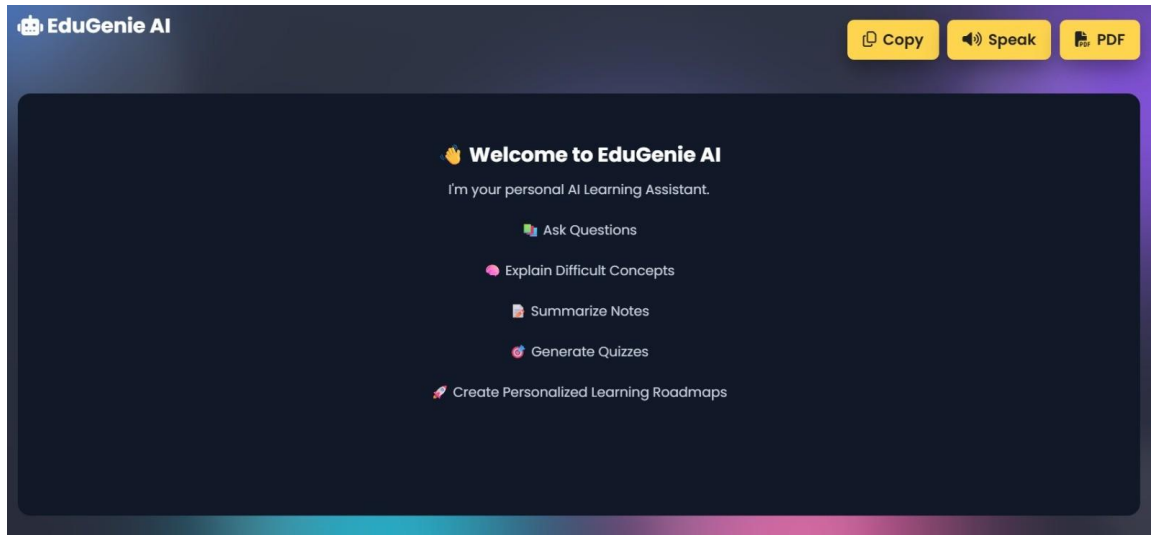


Figure: EduGenie AI welcome screen with Copy, Speak, and PDF response actions, confirming the interface renders and behaves as expected.

Milestone 5: Testing and Verification

In this milestone, each of EduGenie's core features was manually tested to confirm correct behaviour end-to-end, from user input through AI generation to the rendered response. **Activity 5.1:**

Feature Testing

- Question Answering: Verified that academic questions return accurate, context-rich answers from Gemini 2.5 Flash.
- Quiz Generation: Confirmed that topic-specific multiple-choice quizzes are generated along with a correct answer key.
- Learning Recommendation: Checked that structured, multi-level learning roadmaps are produced for a given topic.
- Image Analysis: Uploaded educational images and confirmed Gemini's vision capability correctly interprets and explains their content.
- PDF Analysis: Uploaded lecture note PDFs and confirmed that PyMuPDF-extracted text is used correctly as context for question answering.

Activity 5.2: API Testing with Postman

Backend routes were independently tested using Postman to validate request and response formats, error handling for missing or invalid inputs, and correct status codes before final integration with the frontend.

Milestone 5: Testing and Local Deployment (contd.)

In addition to feature-level testing, EduGenie's deployment readiness was verified through API testing, version control, and a final full-system check before the project was considered complete.

Activity 5.3: API Testing Using Postman

The EduGenie application was tested using Postman to verify that all backend API endpoints function correctly. Each route — question answering, topic explanation, quiz generation, learning recommendation, image analysis, and PDF analysis — was exercised with a range of valid and invalid inputs to confirm accurate request processing and response generation. Successful testing confirmed that the FastAPI backend correctly communicates with the Gemini and LaMini-Flan-T5 services and returns well-structured educational responses. Error handling and response validation, including missing files and malformed requests, were also verified during this stage.

Activity 5.4: GitHub Repository and Project Organisation

Version control was managed using Git and GitHub throughout development. The complete source code, project structure, and configuration files were committed to a GitHub repository for secure storage and collaboration, with the .env file excluded via .gitignore to protect the Gemini API key. The final submission was organised into clearly separated folders for project documentation, project demonstration, and the application's code files, making it straightforward for reviewers to locate the source code, services, static assets, and templates.

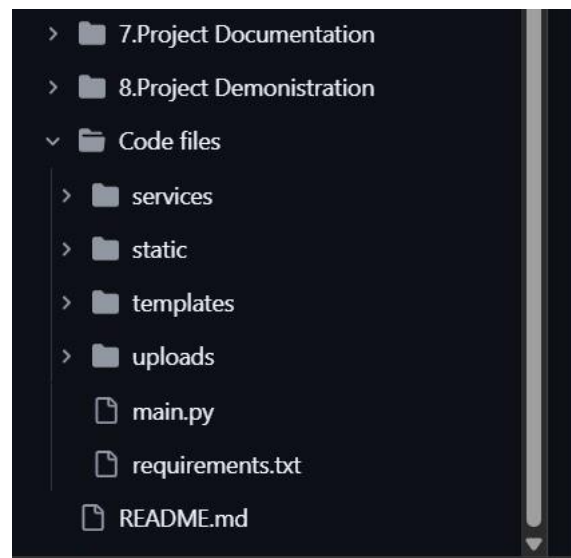


Figure: Final submission folder structure — Project Documentation, Project Demonstration, and Code files containing services, static, templates, uploads, main.py, requirements.txt, and README.md.

Activity 5.5: Project Verification

The complete EduGenie application was verified by testing all implemented modules together — Question Answering, Topic Explanation, Summary Generation, Quiz Generation, Learning Recommendation, Image Analysis, and PDF Analysis. Generated responses were validated for correctness, relevance, and proper Markdown formatting. The successful execution of every module confirmed that EduGenie meets its intended project objectives and functional requirements in the local development environment, and is ready for the enhancements described in the Future Scope section.

Future Scope

While EduGenie already provides a comprehensive suite of AI-powered educational tools, several enhancements have been identified for future development:

- Adding user authentication so students can save and revisit their learning history and generated quizzes.
- Introducing multi-language support to make EduGenie accessible to a wider range of learners.
- Expanding PDF analysis to support multi-document context and larger academic materials.
- Adding progress tracking and analytics dashboards for students and educators.
- Deploying EduGenie to a cloud platform for public accessibility beyond local development.
- Migrating from the deprecated google-generativeai package to the newer google-genai SDK.

Conclusion

EduGenie is an AI-powered educational assistant built with FastAPI and integrated with Google Gemini 2.5 Flash and LaMini-Flan-T5 to deliver intelligent, multi-modal learning support. Through its question-answering, quiz generation, learning recommendation, image analysis, and PDF interaction features, EduGenie streamlines the way students engage with academic content.

The project followed a structured development process, from selecting and configuring the appropriate AI models, to building modular backend services, developing a responsive frontend, and thoroughly testing each feature. This milestone-driven approach ensured that EduGenie evolved into a reliable, modular, and extensible learning assistant.

Looking ahead, EduGenie has significant potential for growth, including multi-language support, user authentication, progress tracking, and cloud deployment, positioning it to evolve from an academic project into a comprehensive AI-driven learning companion for students and educators alike.

Project Outcome

The EduGenie AI Learning Assistant was successfully developed and tested using FastAPI, Google Gemini AI, and LaMini-Flan-T5. The application provides multiple educational features, including Question Answering, Topic Explanation, Summary Generation, Quiz Generation, Learning Recommendations, Image Analysis, and PDF Analysis, all accessible through a single responsive web interface. The project demonstrates the effective integration of Generative AI with a modern web application to enhance the learning experience, and all core functionalities were tested successfully in the local development environment prior to submission.

Future Enhancements

Although EduGenie successfully implements all of its required educational features, several enhancements have been identified to extend the platform further:

- Adding user authentication so students can save and revisit their learning history and generated quizzes.
- Introducing multi-language support to make EduGenie accessible to a wider range of learners.
- Expanding PDF analysis to support multi-document context and larger academic materials.
- Adding voice-based interaction for hands-free question answering and note review.
- Adding student progress tracking and analytics dashboards for both students and educators.
- Deploying EduGenie to a cloud platform for public accessibility beyond local development.
- Incorporating additional AI models and database integration to improve scalability and provide a richer, more personalised learning history for each user.
- Migrating from the deprecated google-generativeai package to the newer google-genai SDK.

Final Conclusion

EduGenie is an AI-powered educational platform that demonstrates the practical application of Generative AI in modern learning systems. By integrating Google Gemini AI and LaMini-Flan-T5 with a FastAPI backend and a responsive HTML, CSS, and JavaScript frontend, the application provides intelligent educational assistance through interactive features such as question answering, topic explanation, summarisation, quizzes, learning recommendations, image analysis, and PDF-based learning. The project followed a structured, milestone-driven development process — from model selection and architecture design, through core AI service development, backend and frontend implementation, to thorough feature and API testing — ensuring that EduGenie evolved into a reliable, modular, and extensible learning assistant. The project highlights the potential of Generative AI in improving personalised education and serves as a strong foundation for future AI-driven educational applications.